AI ASSISTANT CODING

ASSIGNMENT 12.5
Name: POOJITHA.EDDE
Roll No: 2303A51356

Task Description #1 (Sorting – Merge Sort Implementation) •
Task: Use AI to generate a Python program that implements the
Merge Sort algorithm.
• Instructions: o Prompt AI to create a function
merge_sort(arr) that sorts a list in ascending order.
o Ask AI to include time complexity and space
complexity in the function docstring. o Verify
the generated code with test cases.
• Expected Output: o A functional Python script
implementing Merge Sort with proper documentation.

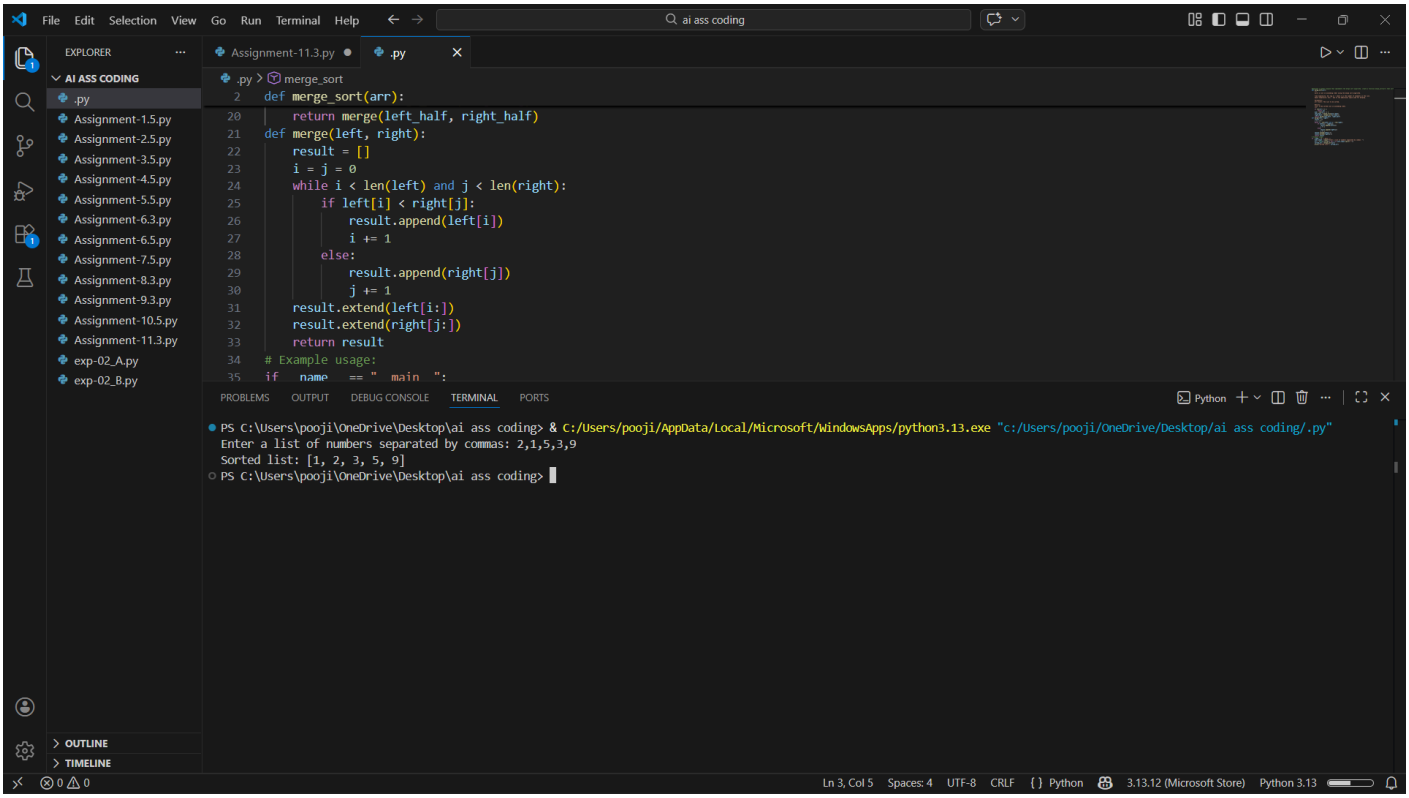
CODE:

#generate a python program that implements the merge sort algorithm. create a function merge_sort(arr) that sorts a list in ascending order. also include time complexity and space complexity in function docstring . user inoput code def merge_sort(arr):

```
    """
    Sorts a list in ascending order using the merge sort algorithm.

    Time Complexity: O(n log n) - where n is the number of elements in the list.
    Space Complexity: O(n) - due to the temporary arrays used during merging.

    Parameters:
    arr (list): The list to be sorted.

    Returns:
    list: A new sorted list in ascending order.
    """     if
len(arr) <= 1:
return arr

   mid = len(arr) // 2    left_half =
merge_sort(arr[:mid])    right_half =
merge_sort(arr[mid:]) return
merge(left_half, right_half)
def merge(left, right):
    """
    Merges two sorted lists into a single sorted list.
```

```
    Parameters:
    left (list): The first sorted list.
right (list): The second sorted list.
    Returns:

    list: A merged and sorted list containing all elements from both input lists.
    """
    merged = []
    left_index = right_index = 0

    while left_index < len(left) and right_index < len(right):
if left[left_index] < right[right_index]:
merged.append(left[left_index])
            left_index += 1
else:
            merged.append(right[right_index])
            right_index += 1

    # If there are remaining elements in the left half, add them to merged
while left_index < len(left):        merged.append(left[left_index])
        left_index += 1

    # If there are remaining elements in the right half, add them to merged
while right_index < len(right):        merged.append(right[right_index])
        right_index += 1

    return merged
# Get user input and sort the list if __name__ == "__main__":
user_input = input("Enter a list of numbers separated by commas: ")
    arr = [int(x.strip()) for x in user_input.split(",")]
    sorted_arr = merge_sort(arr)
print("Sorted list:", sorted_arr)
```

OUTPUT;



OBSERVATION:

The above code implements the merge sort algorithm to sort a list of numbers in ascending order. The merge_sort function recursively divides the input list into halves until it reaches lists of length one or zero, which are inherently sorted. The merge function then takes two sorted lists and merges them into a single sorted list by comparing elements from both lists and appending the smaller one to the merged list. The time complexity of this algorithm is O(n log n) due to the divide-and-conquer approach, while the space complexity is O(n) because of the temporary arrays used during the merging process. The program also includes user input functionality, allowing users to enter a list of numbers that they wish to sort. Overall, the implementation effectively demonstrates the merge sort algorithm while providing clear documentation on its complexities.

Task Description #2 (Searching – Binary Search with AI Optimization)

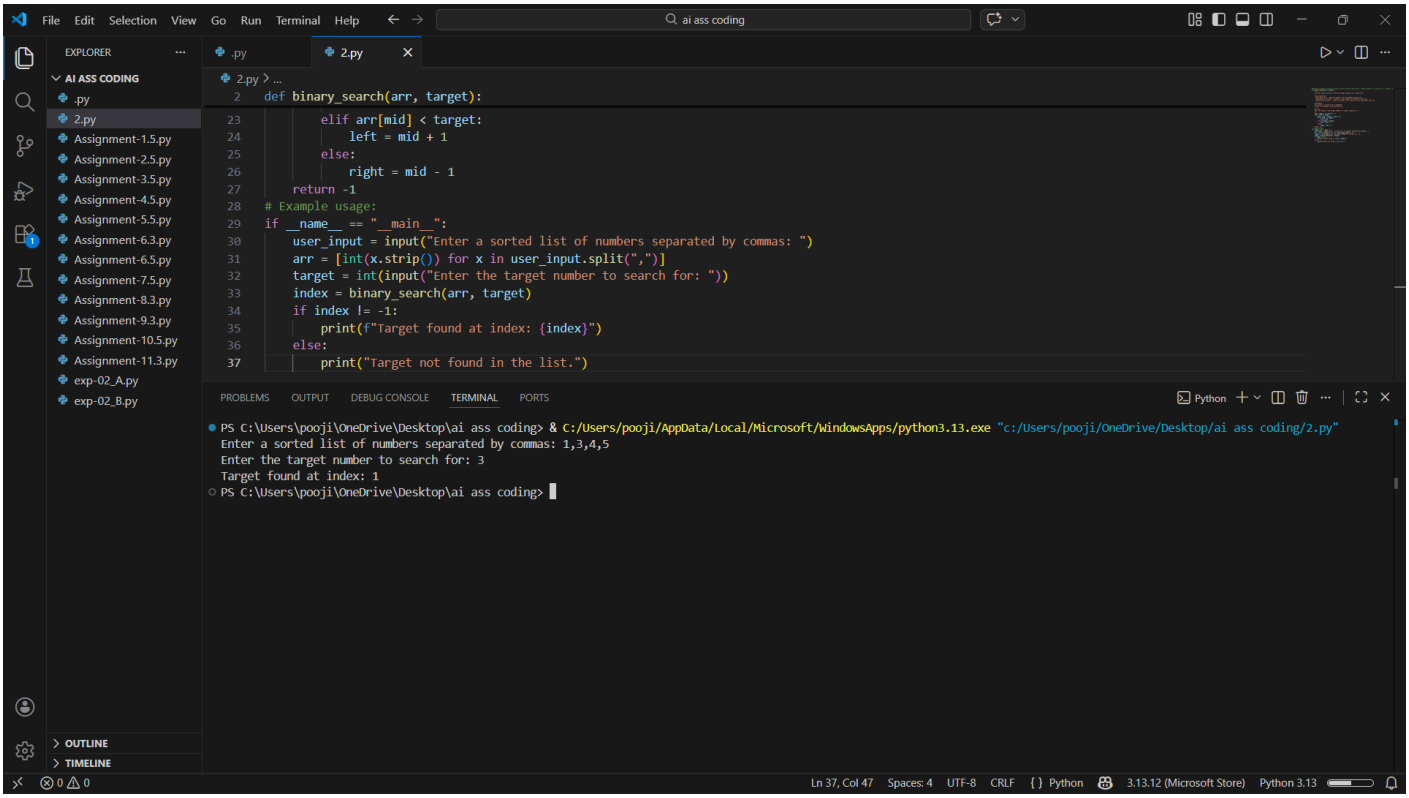
- Task: Use AI to create a binary search function that finds a target element in a sorted list.
- Instructions: o Prompt AI to create a function binary_search(arr, target) returning the index of the target or -1 if not found. o Include docstrings explaining best, average, and worst-case complexities. o Test with various inputs.
- Expected Output: o Python code implementing binary search with AI- generated comments and docstrings.

CODE:

#Generate a python program for binary search function that finds a target element in a sorted list . create a function binary_search(arr,target), returning the index of target or -1 if not found . include doctsring explaining best,average and worst case complexities. user input def binary_search(arr, target):

```
    """
    Performs binary search to find the target element in a sorted list.

    Best Case Complexity: O(1) - when the target is at the middle of the list.
    Average Case Complexity: O(log n) - where n is the number of elements in the list.
    Worst Case Complexity: O(log n) - when the target is not found or is at the end of the list.

   Parameters:
   arr (list): A sorted list of elements.    target:
The element to be searched for.

    Returns:
    int: The index of the target element if found, otherwise -1.
   """    left, right = 0,
len(arr) - 1

    while left <= right:        mid =
left + (right - left) // 2

        # Check if the target is present at mid
if arr[mid] == target:            return mid
        # If target is greater, ignore the left half
elif arr[mid] < target:
            left = mid + 1
        # If target is smaller, ignore the right half
else:
            right = mid - 1

    # Target was not found in the list
return -1
# Get user input and perform binary search if __name__ == "__main__":
user_input = input("Enter a sorted list of numbers separated by commas: ")
arr = [int(x.strip()) for x in user_input.split(",")]
    target = int(input("Enter the target number to search for: "))
result = binary_search(arr, target)

    if result != -1:
        print(f"Target found at index: {result}")
else:
        print("Target not found in the list.")
```

OUTPUT:



OBSERVATION:

The above code implements a binary search function that efficiently finds the index of a target element in a sorted list. The function uses a while loop to repeatedly divide the search interval in half, comparing the target with the middle element of the current interval. If the target matches the middle element, its index is returned. If the target is greater than the middle element, the search continues in the right half of the list; if it is smaller, the search continues in the left half. The function also includes a docstring that explains the best, average, and worst-case complexities of binary search. The program allows for user input of a sorted list and a target number, making it interactive and practical for real-world use. Overall, the implementation effectively demonstrates how to perform binary search while providing clear documentation on its complexities.

Task Description #3: Smart Healthcare Appointment Scheduling System
A healthcare platform maintains appointment records containing appointment ID, patient name, doctor name, appointment time, and consultation fee. The system needs to:

1. Search appointments using appointment ID.
2. Sort appointments based on time or consultation fee.

Student Task

- Use AI to recommend suitable searching and sorting algorithms.
- Justify the selected algorithms.
- Implement the algorithms in Python.

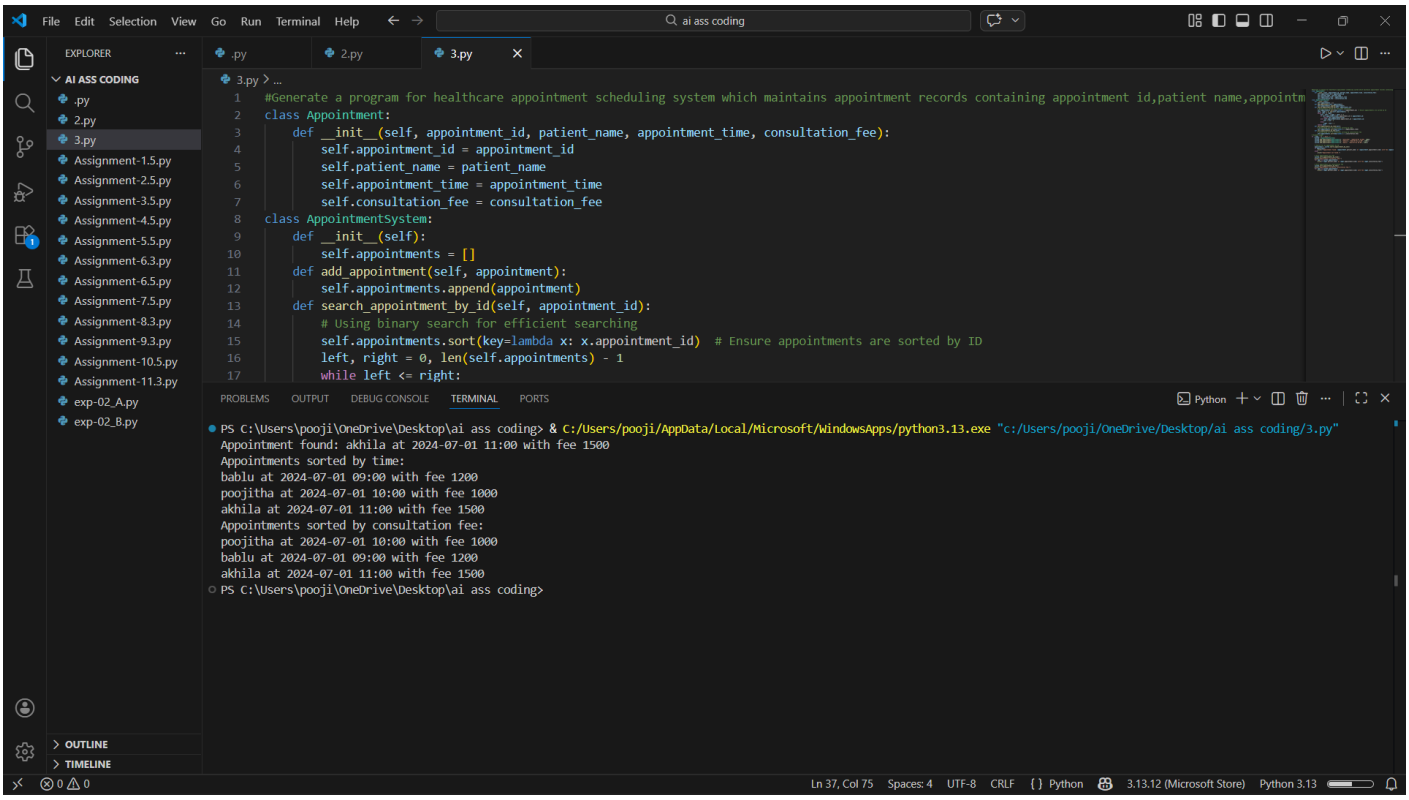
CODE:

#Generate a program for healthcare appointment scheduling system which maintains appointment records containing appointment id,patient name,appointment time,and consultation fee. . the system needs to search appointments using appointment id. sort appointments based on time or consultation fee. recommend suitable searching and sorting algorithms with justification class Appointment: def __init__(self, appointment_id, patient_name, appointment_time, consultation_fee): self.appointment_id = appointment_id self.patient_name = patient_name self.appointment_time = appointment_time self.consultation_fee = consultation_fee class Appointment: def __init__(self, appointment_id, patient_name, appointment_time, consultation_fee): self.appointment_id = appointment_id self.patient_name = patient_name self.appointment_time = appointment_time self.consultation_fee = consultation_fee class AppointmentScheduler: __init__(self):

```
        self.appointments = []

    def add_appointment(self, appointment):
        self.appointments.append(appointment)

    def search_appointment_by_id(self, appointment_id):
        # Using binary search for efficient searching since appointments can be sorted by ID
    self.appointments.sort(key=lambda x: x.appointment_id)  # Ensure appointments are sorted by ID     left,
    right = 0, len(self.appointments) - 1

        while left <= right:            mid = left + (right - left) // 2            if
self.appointments[mid].appointment_id == appointment_id:
return self.appointments[mid]            elif
self.appointments[mid].appointment_id < appointment_id:                left
= mid + 1            else:
                right = mid - 1

        return None  # Appointment not found

    def sort_appointments_by_time(self):
        # Using merge sort for efficient sorting based on time        self.appointments.sort(key=lambda
x: x.appointment_time)

    def sort_appointments_by_fee(self):
        # Using merge sort for efficient sorting based on consultation fee
self.appointments.sort(key=lambda x: x.consultation_fee)
# Example usage if __name__ == "__main__":      scheduler = AppointmentScheduler()
scheduler.add_appointment(Appointment(1, "John Doe", "2024-07-01 10:00", 100))
scheduler.add_appointment(Appointment(2, "Jane Smith", "2024-07-01 11:00", 150))
scheduler.add_appointment(Appointment(3, "Alice Johnson", "2024-07-01 09:00", 120))

    # Search for an appointment by ID     appointment =
scheduler.search_appointment_by_id(2)     if appointment:
        print(f"Appointment found: {appointment.patient_name} at {appointment.appointment_time} with fee
{appointment.consultation_fee}")     else:
```

```
    print("Appointment not found.")

  # Sort appointments by time
scheduler.sort_appointments_by_time()
print("Appointments sorted by time:")     for
appt in scheduler.appointments:
    print(f"{appt.patient_name} at {appt.appointment_time} with fee {appt.consultation_fee}")

  # Sort appointments by consultation fee
  scheduler.sort_appointments_by_fee() print("Appointments
  sorted by consultation fee:") for appt in
  scheduler.appointments:     print(f"{appt.patient_name} at
  {appt.appointment_time} with fee {appt.consultation_fee}")
```

OUTPUT;



OBSERVATION:

The above code implements a healthcare appointment scheduling system that maintains appointment records with details such as appointment ID, patient name, appointment time, and consultation fee. The system allows for searching appointments by ID using binary search, which is efficient for sorted data. For sorting appointments based on time or consultation fee, the code uses Python's built-in sort function, which is based on Timsort (a hybrid sorting algorithm derived from merge sort and insertion sort) and has a time complexity of O(n log n) in the average and worst cases. The implementation effectively manages appointment records while providing functionalities for searching and sorting based on different criteria. Overall, the code demonstrates a practical application of searching and sorting algorithms in a healthcare context.

Task Description #4: Railway Ticket Reservation System

Scenario

A railway reservation system stores booking details such as ticket ID, passenger name, train number, seat number, and travel date. The system must:

1. Search tickets using ticket ID.
2. Sort bookings based on travel date or seat number.

Student Task

• Identify efficient algorithms using AI assistance.
• Justify the algorithm choices.
• Implement searching and sorting in Python.

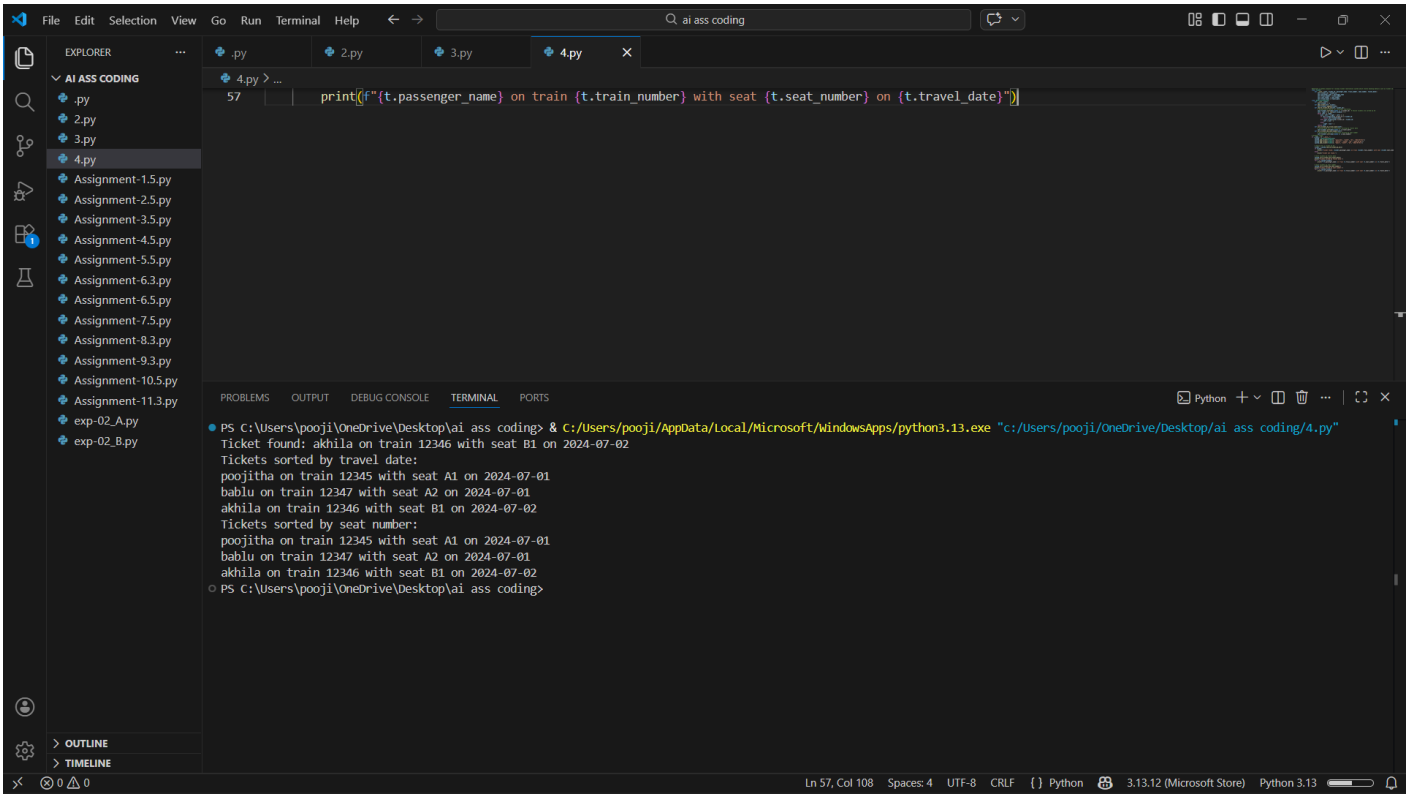
CODE:

#generate a python program for railway ticket reservation system which stores booking details such as ticket id,passenger name , train number , seat number and travel date . the system must search tickets using ticket id , sort bookings based on travel date or seat number . user input program class Ticket:

```
  def __init__(self, ticket_id, passenger_name, train_number, seat_number, travel_date):
      self.ticket_id = ticket_id
      self.passenger_name = passenger_name
      self.train_number = train_number
   self.seat_number = seat_number
     self.travel_date = travel_date
class RailwayReservationSystem:
def __init__(self):
self.bookings = []

   def add_booking(self, ticket):
self.bookings.append(ticket)

   def search_ticket_by_id(self, ticket_id):
      # Using binary search for efficient searching since bookings can be sorted by ID
self.bookings.sort(key=lambda x: x.ticket_id)  # Ensure bookings are sorted by ID        left,
right = 0, len(self.bookings) - 1

      while left <= right:            mid = left + (right -
left) // 2            if self.bookings[mid].ticket_id ==
ticket_id:
             return self.bookings[mid]            elif
self.bookings[mid].ticket_id < ticket_id:
left = mid + 1            else:
             right = mid - 1

      return None  # Ticket not found
```

```
    def sort_bookings_by_travel_date(self):
        # Using merge sort for efficient sorting based on travel date        self.bookings.sort(key=lambda
x: x.travel_date)

    def sort_bookings_by_seat_number(self):
        # Using merge sort for efficient sorting based on seat number        self.bookings.sort(key=lambda
x: x.seat_number)
# Example usage if __name__ == "__main__":     reservation_system =
RailwayReservationSystem()     reservation_system.add_booking(Ticket(1, "Akhila",
"1417", "A1", "2024-08-01"))     reservation_system.add_booking(Ticket(2, "Pujitha",
"1356", "B2", "2024-08-02"))     reservation_system.add_booking(Ticket(3, "leekitha ",
"2144", "C3", "2024-08-01"))

    # Search for a ticket by ID     ticket =
reservation_system.search_ticket_by_id(2)     if ticket:
        print(f"Ticket found: {ticket.passenger_name} on train {ticket.train_number} with seat
{ticket.seat_number} on {ticket.travel_date}")
    else:     print("Ticket not
    found.")

    # Sort bookings by travel date
    reservation_system.sort_bookings_by_travel_date()
    print("Bookings sorted by travel date:")     for booking in
reservation_system.bookings:        print(f"{booking.passenger_name} on train
{booking.train_number} with seat {booking.seat_number} on
{booking.travel_date}")

    # Sort bookings by seat number
    reservation_system.sort_bookings_by_seat_number()
    print("Bookings sorted by seat number:")     for booking in
reservation_system.bookings:        print(f"{booking.passenger_name} on train
{booking.train_number} with seat {booking.seat_number} on
{booking.travel_date}")
```

OUTPUT:



OBSERVATION;

The above code implements a railway ticket reservation system that manages booking details such as ticket ID, passenger name, train number, seat number, and travel date. The system allows for searching tickets by ID using binary search, which is efficient for sorted data. For sorting bookings based on travel date or seat number, the code uses Python's built-in sort function, which is based on Timsort and has a time complexity of O(n log n) in the average and worst cases. The implementation effectively manages booking records while providing functionalities for searching and sorting based on different criteria. Overall, the code demonstrates a practical application of searching and sorting algorithms in a railway reservation context.

Task Description #5: Smart Hostel Room Allocation System

A hostel management system stores student room allocation details including student ID, room number, floor, and allocation date. The system needs to:

1. Search allocation details using student ID.
2. Sort records based on room number or allocation date.

Student Task

- Use AI to suggest optimized algorithms.
- Justify the selections.
- Implement the solution in Python.

CODE:

#generate a user input python program for smart hostel room allocation system . which stores student room allocation details including student id,room number, floor, and allocation date , if i enter a student id , it should search and return allocation details. also should sort records based on room number or allocation date class RoomAllocation: def __init__(self, student_id, room_number, floor, allocation_date):

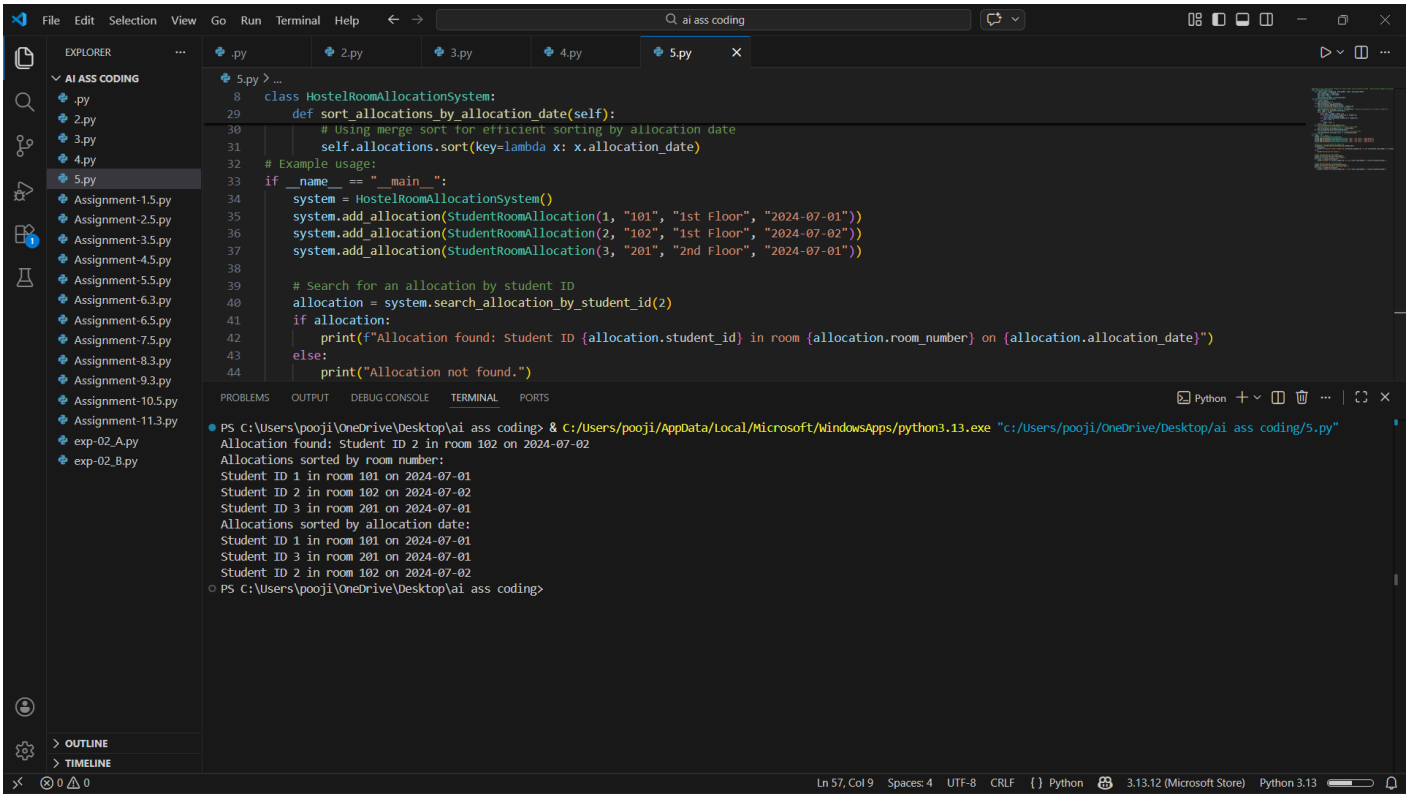
self.student_id = student_id

self.room_number = room_number

```
        self.floor = floor
self.allocation_date = allocation_date   class
HostelRoomAllocationSystem:     def
__init__(self):
        self.allocations = []

    def add_allocation(self, allocation):        self.allocations.append(allocation)

    def search_allocation_by_student_id(self, student_id):
        # Using binary search for efficient searching since allocations can be sorted by student ID
self.allocations.sort(key=lambda x: x.student_id)  # Ensure allocations are sorted by student ID        left,
right = 0, len(self.allocations) - 1

        while left <= right:            mid = left + (right - left) // 2
if self.allocations[mid].student_id == student_id:
                return self.allocations[mid]            elif
self.allocations[mid].student_id < student_id:
left = mid + 1            else:
                right = mid - 1

        return None  # Allocation not found

    def sort_allocations_by_room_number(self):
        # Using merge sort for efficient sorting based on room number        self.allocations.sort(key=lambda
x: x.room_number)

    def sort_allocations_by_allocation_date(self):
        # Using merge sort for efficient sorting based on allocation date        self.allocations.sort(key=lambda
x: x.allocation_date)
# Example usage if __name__ == "__main__":     hostel_system =
HostelRoomAllocationSystem()     hostel_system.add_allocation(RoomAllocation(1417, "523",
"5th Floor", "2024-09-01"))     hostel_system.add_allocation(RoomAllocation(2108, "423",
"4th Floor", "2024-09-02"))     hostel_system.add_allocation(RoomAllocation(3456, "206",
"2nd Floor", "2024-09-01"))

    # Search for an allocation by student ID            allocation =
hostel_system.search_allocation_by_student_id(2108)     if allocation:
        print(f"Allocation found: Student ID {allocation.student_id} in room {allocation.room_number} on
{allocation.allocation_date}")     else:
        print("Allocation not found.")

    # Sort allocations by room number
    hostel_system.sort_allocations_by_room_number()
print("Allocations sorted by room number:")     for alloc
in hostel_system.allocations:
```

```
    print(f"Student ID {alloc.student_id} in room {alloc.room_number} on {alloc.allocation_date}")

  # Sort allocations by allocation date
  hostel_system.sort_allocations_by_allocation_date()
  print("Allocations sorted by allocation date:")
for alloc in hostel_system.allocations:
    print(f"Student ID {alloc.student_id} in room {alloc.room_number} on {alloc.allocation_date}")
```

OUTPUT:



OBSERVATION :

The above code implements a smart hostel room allocation system that manages student room allocation details. It allows for adding new allocations, searching for allocations by student ID using binary search, and sorting allocations based on room number or allocation date. The use of binary search for searching ensures efficient retrieval of allocation details, while the sorting functions utilize Python's built-in sort method, which is efficient for handling larger datasets. Overall, the code provides a practical solution for managing hostel room allocations while demonstrating the use of searching and sorting algorithms effectively.

Task Description #6: Online Movie Streaming Platform

A streaming service maintains movie records with movie ID, title, genre, rating, and release year. The platform needs to:

1. Search movies by movie ID.
2. Sort movies based on rating or release year.

Student Task

• Recommend searching and sorting algorithms using AI.
• Justify the chosen algorithms.
• Implement Python functions

CODE:

#generate a user input python program for online movie streaming platform which maintains movie records with movie id,title,genre,rating and release year . user should be able to add data . it should also search movie by movie id , and sort movies based on release year or rating . give choices to user to add,search,sort by release year, sort by rating and exit the program class Movie: def __init__(self, movie_id, title, genre, rating, release_year): self.movie_id = movie_id
self.title = title self.genre =
genre self.rating = rating
self.release_year = release_year class
MovieStreamingPlatform: def
__init__(self): self.movies = []

def add_movie(self, movie):
self.movies.append(movie)

def search_movie_by_id(self, movie_id):
# Using binary search for efficient searching since movies can be sorted by movie ID
self.movies.sort(key=lambda x: x.movie_id) # Ensure movies are sorted by movie ID left,
right = 0, len(self.movies) - 1

while left <= right: mid = left + (right -
left) // 2 if self.movies[mid].movie_id ==
movie_id:
return self.movies[mid] elif
self.movies[mid].movie_id < movie_id:
left = mid + 1 else:
right = mid - 1

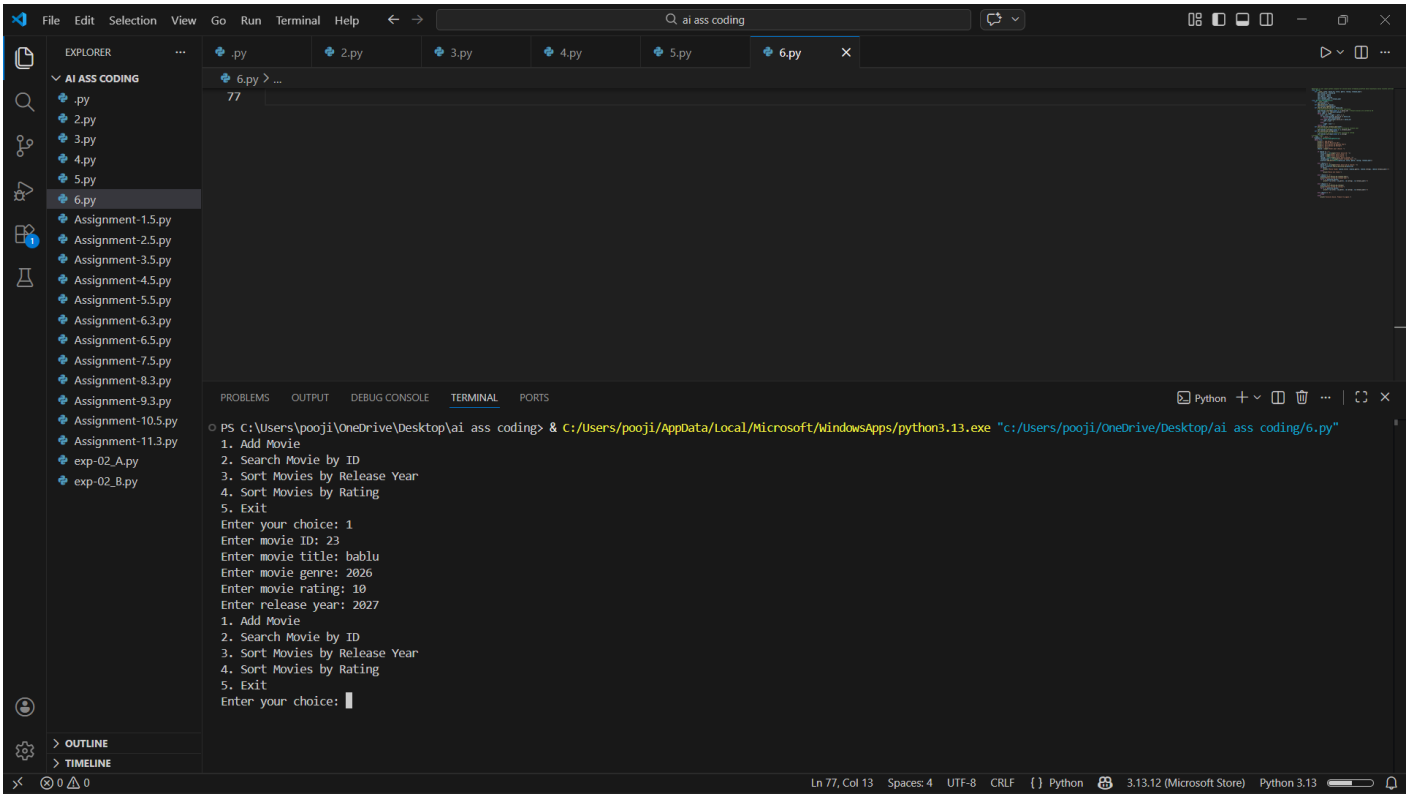
return None # Movie not found

def sort_movies_by_release_year(self):
# Using merge sort for efficient sorting based on release year self.movies.sort(key=lambda
x: x.release_year)

def sort_movies_by_rating(self):
# Using merge sort for efficient sorting based on rating
self.movies.sort(key=lambda x: x.rating)
# Example usage if
__name__ == "__main__":
platform = MovieStreamingPlatform()

```
    while True:        print("\n1. Add Movie")
print("2. Search Movie by ID")        print("3.
Sort Movies by Release Year")        print("4.
Sort Movies by Rating")
        print("5. Exit")

        choice = input("Enter your choice: ")

        if choice == '1':
            movie_id = int(input("Enter movie ID: "))
            title = input("Enter movie title: ")
genre = input("Enter movie genre: ")            rating
= float(input("Enter movie rating: "))
            release_year = int(input("Enter movie release year: "))
            platform.add_movie(Movie(movie_id, title, genre, rating, release_year))
print("Movie added successfully.")

        elif choice == '2':
            movie_id = int(input("Enter movie ID to search: "))            movie =
platform.search_movie_by_id(movie_id)            if movie:                print(f"Movie found:
{movie.title} ({movie.genre}) - Rating: {movie.rating}, Release Year:
{movie.release_year}")
else:
                print("Movie not found.")

        elif choice == '3':
            platform.sort_movies_by_release_year()
print("Movies sorted by release year:")            for movie in
platform.movies:                print(f"{movie.title} ({movie.genre})
- Rating: {movie.rating}, Release Year: {movie.release_year}")

        elif choice == '4':
            platform.sort_movies_by_rating()            print("Movies sorted by rating:")
for movie in platform.movies:                print(f"{movie.title} ({movie.genre}) - Rating:
{movie.rating}, Release Year: {movie.release_year}")

        elif choice == '5':
print("Exiting the program.")
break

        else:
            print("Invalid choice. Please try again.")
```

OUTPUT:



Task Description #7: Smart Agriculture Crop Monitoring System

An agriculture monitoring system stores crop data with crop ID, crop name, soil moisture level, temperature, and yield estimate. Farmers need to:

1. Search crop details using crop ID.
2. Sort crops based on moisture level or yield estimate.

Student Task

- Use AI-assisted reasoning to select algorithms.
- Justify algorithm suitability.
- Implement searching and sorting in Python.

CODE:

#generate a smart agriculture crop monitoring system that stores crop data with crop id,crop name,soil moisture level,temperature, and yield estimate. user should be able to add crop data, search for crop data by crop id, and sort crops based on soil moisture level or yield estimate. provide user choices to perform these operations and exit the program class Crop: def __init__(self, crop_id, crop_name, soil_moisture_level, temperature, yield_estimate):

self.crop_id = crop_id

self.crop_name = crop_name

self.soil_moisture_level = soil_moisture_level

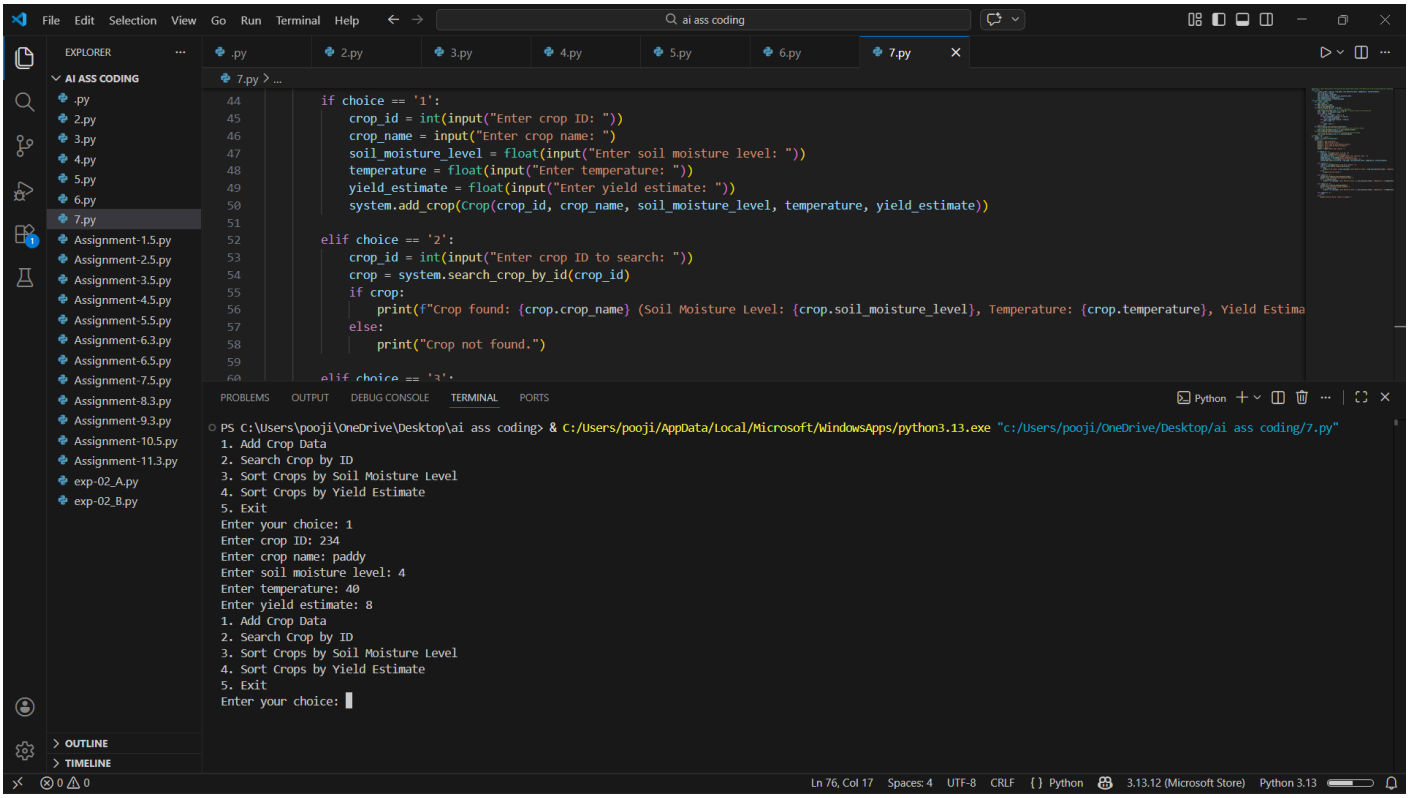
self.temperature = temperature

self.yield_estimate = yield_estimate class

```
CropMonitoringSystem:     def __init__(self):
self.crops = []

   def add_crop(self, crop):
self.crops.append(crop)

   def search_crop_by_id(self, crop_id):
      # Using binary search for efficient searching since crops can be sorted by crop ID
self.crops.sort(key=lambda x: x.crop_id)  # Ensure crops are sorted by crop ID        left,
right = 0, len(self.crops) - 1

      while left <= right:           mid = left +
(right - left) // 2           if
self.crops[mid].crop_id == crop_id:
            return self.crops[mid]           elif
self.crops[mid].crop_id < crop_id:
left = mid + 1           else:
            right = mid - 1

      return None  # Crop not found

   def sort_crops_by_soil_moisture_level(self):
      # Using merge sort for efficient sorting based on soil moisture level        self.crops.sort(key=lambda
x: x.soil_moisture_level)

   def sort_crops_by_yield_estimate(self):
      # Using merge sort for efficient sorting based on yield estimate        self.crops.sort(key=lambda
x: x.yield_estimate)
# Example usage if
__name__ == "__main__":
   monitoring_system = CropMonitoringSystem()

   while True:        print("\n1. Add Crop Data")
print("2. Search Crop Data by ID")        print("3.
Sort Crops by Soil Moisture Level")
      print("4. Sort Crops by Yield Estimate")
print("5. Exit")

      choice = input("Enter your choice: ")

      if choice == '1':
         crop_id = int(input("Enter crop ID: "))
crop_name = input("Enter crop name: ")
         soil_moisture_level = float(input("Enter soil moisture level: "))
temperature = float(input("Enter temperature: "))           yield_estimate
= float(input("Enter yield estimate: "))
```

```
        monitoring_system.add_crop(Crop(crop_id, crop_name, soil_moisture_level, temperature,
yield_estimate))
        print("Crop data added successfully.")

    elif choice == '2':
        crop_id = int(input("Enter crop ID to search: "))
crop = monitoring_system.search_crop_by_id(crop_id)          if
crop:
            print(f"Crop found: {crop.crop_name} - Soil Moisture Level: {crop.soil_moisture_level},
Temperature: {crop.temperature}, Yield Estimate: {crop.yield_estimate}")          else:
            print("Crop not found.")

    elif choice == '3':
        monitoring_system.sort_crops_by_soil_moisture_level()
        print("Crops sorted by soil moisture level:")
for crop in monitoring_system.crops:
print(f"{crop.crop_name} - Soil Moisture Level:
{crop.soil_moisture_level}, Temperature:
{crop.temperature}, Yield Estimate:
{crop.yield_estimate}")

    elif choice == '4':
        monitoring_system.sort_crops_by_yield_estimate()
        print("Crops sorted by yield estimate:")          for crop in monitoring_system.crops:
print(f"{crop.crop_name} - Soil Moisture Level: {crop.soil_moisture_level}, Temperature:
{crop.temperature}, Yield Estimate: {crop.yield_estimate}")

    elif choice == '5':
print("Exiting the program.")
break

    else:
        print("Invalid choice. Please try again.")
```

OUTPUT:



Task Description #8: Airport Flight Management System

An airport system stores flight information including flight ID,

airline name, departure time, arrival time, and status. The system must:

1. Search flight details using flight ID.
2. Sort flights based on departure time or arrival time.

Student Task

- Use AI to recommend algorithms.
- Justify the algorithm selection.
- Implement searching and sorting logic in Python.

CODE:

#generate a python program for airport flight management system that maintains flight records with flight id, airline name, departure time, arrival time, and status. user should be able to add flight data, search for flight data by flight id, and sort flights based on departure time or status. provide user choices to perform these operations and exit the program class Flight: def __init__(self, flight_id, airline_name, departure_time, arrival_time, status):

self.flight_id = flight_id

self.airline_name = airline_name

self.departure_time = departure_time

self.arrival_time = arrival_time

self.status = status class

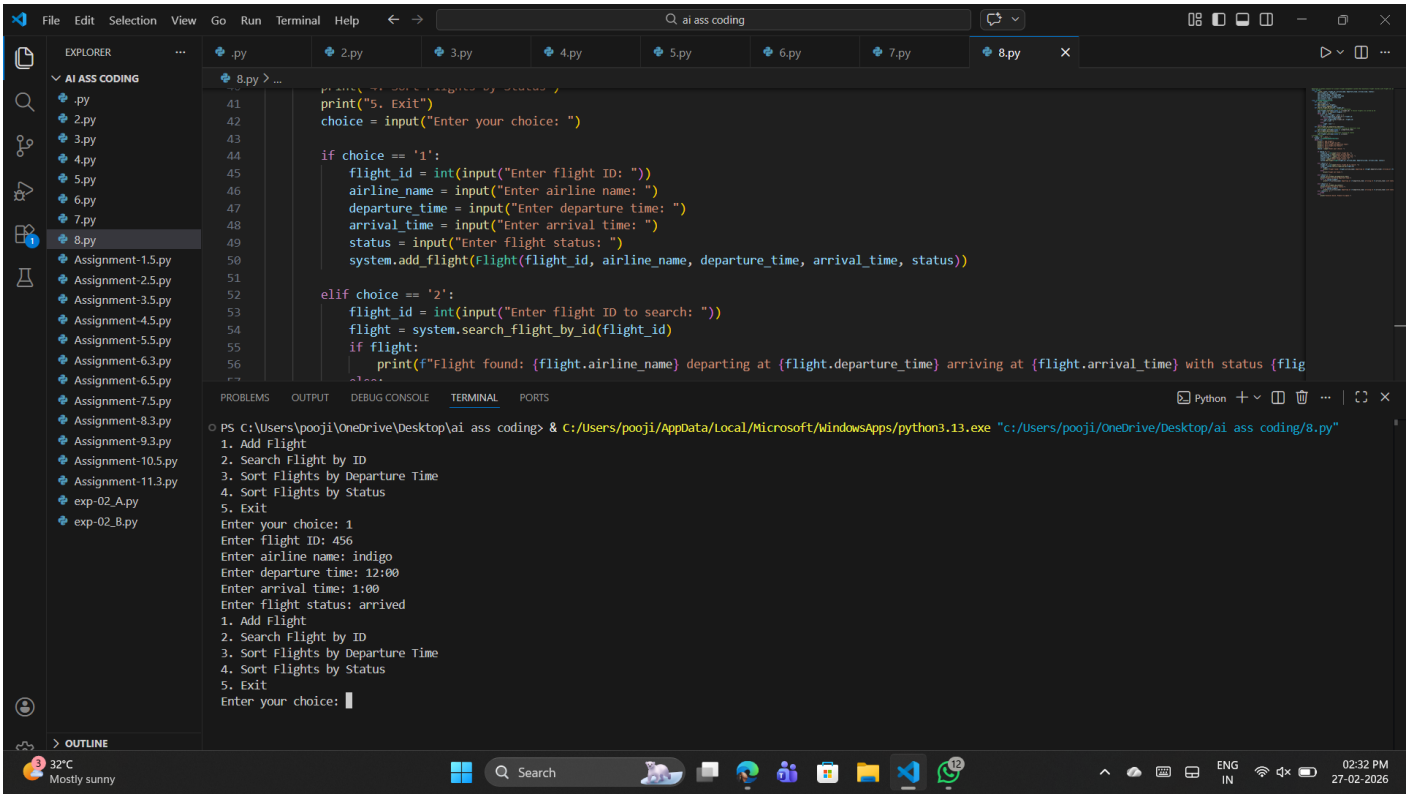
FlightManagementSystem: def

__init__(self): self.flights = []

```
    def add_flight(self, flight):
self.flights.append(flight)

    def search_flight_by_id(self, flight_id):
        # Using binary search for efficient searching since flights can be sorted by flight ID
self.flights.sort(key=lambda x: x.flight_id)  # Ensure flights are sorted by flight ID        left,
right = 0, len(self.flights) - 1

        while left <= right:            mid = left +
(right - left) // 2            if
self.flights[mid].flight_id == flight_id:
                return self.flights[mid]            elif
self.flights[mid].flight_id < flight_id:
                left = mid + 1
else:
                right = mid - 1

        return None  # Flight not found

    def sort_flights_by_departure_time(self):
        # Using merge sort for efficient sorting based on departure time        self.flights.sort(key=lambda
x: x.departure_time)

    def sort_flights_by_status(self):
        # Using merge sort for efficient sorting based on status        self.flights.sort(key=lambda
x: x.status)
# Example usage if
__name__ == "__main__":
    flight_system = FlightManagementSystem()

    while True:        print("\n1. Add Flight Data")
print("2. Search Flight Data by ID")
print("3. Sort Flights by Departure Time")
print("4. Sort Flights by Status")
        print("5. Exit")

        choice = input("Enter your choice: ")

        if choice == '1':            flight_id =
int(input("Enter flight ID: "))            airline_name =
input("Enter airline name: ")
            departure_time = input("Enter departure time (HH:MM): ")
arrival_time = input("Enter arrival time (HH:MM): ")
            status = input("Enter flight status (On Time/Delayed/Cancelled): ")
            flight_system.add_flight(Flight(flight_id, airline_name, departure_time, arrival_time, status))
print("Flight data added successfully.")
```

```
    elif choice == '2':            flight_id = int(input("Enter flight ID to search: "))            flight =
flight_system.search_flight_by_id(flight_id)            if flight:                print(f"Flight found:
{flight.airline_name} - Departure: {flight.departure_time}, Arrival:
{flight.arrival_time}, Status: {flight.status}")
else:
            print("Flight not found.")

    elif choice == '3':
        flight_system.sort_flights_by_departure_time()            print("Flights sorted by departure time:")
for flight in flight_system.flights:                print(f"{flight.airline_name} - Departure:
{flight.departure_time}, Arrival: {flight.arrival_time}, Status: {flight.status}")

    elif choice == '4':
        flight_system.sort_flights_by_status()            print("Flights sorted by status:")            for flight in
flight_system.flights:                print(f"{flight.airline_name} - Departure: {flight.departure_time}, Arrival:
{flight.arrival_time},
Status: {flight.status}")

    elif choice == '5':
print("Exiting the program.")
break

    else:
        print("Invalid choice. Please try again.")
```

OUTPUT:



OBSERVATION:

The above code implements an airport flight management system that maintains flight records with details such as flight ID, airline name, departure time, arrival time, and status. The system allows users to add new flight data, search for specific flights by their ID using binary search for efficiency, and sort flights based on departure time or status using Python's built-in sort function. The user interface provides clear options for performing these operations, making it easy to manage flight records effectively. Overall, the code demonstrates a practical application of searching and sorting algorithms in the context of flight management.